

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' test (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

MICRO-ADR: Juego Arcade de Naves Espaciales (PROYEC-21)

1. CONTEXTO

Se requiere un juego web estilo arcade de los 80 (Galaga/Asteroids) completamente en HTML5, CSS y JS. El juego debe ejecutarse como archivo estático (sin servidor), lo que activa la restricción crítica del skill: ****ENTORNO LOCAL / file:///****. El requerimiento especifica explícitamente una arquitectura modular con múltiples archivos JS separados (game.js, controls.js, obstacles.js), sin embargo, esta propuesta entra en conflicto directo con la restricción de entorno local donde `<script type='module'>` e `import/export` nativos de ES6 están PROHIBIDOS por incompatibilidad con el protocolo file:///.

2. DECISIÓN ARQUITECTÓNICA

****Monolito Frontend Estático con JavaScript en Namespace Pattern (IIFE + Objeto Global).****

Se adopta una arquitectura de archivo único para el JavaScript (`main.js`) que encapsula todos los módulos del juego mediante el patrón de Namespace/IIFE, garantizando:

- Compatibilidad total con file:/// sin servidor.
- Separación lógica de responsabilidades SIN usar import/export.
- Un único `<canvas>` HTML5 como motor de renderizado (Game Loop con requestAnimationFrame).
- CSS separado en `styles.css` para mantener la separación de concerns visual.

La arquitectura interna de `main.js` se organiza en módulos lógicos encapsulados:

- `GameEngine`: Loop principal, estados del juego, niveles de dificultad.
- `Ship`: Física de la nave (aceleración, frenado sin llegar a cero, rotación).
- `ObstacleManager`: Generación y gestión de los 5 tipos de obstáculos (cometas, asteroides, planetas, OVNIs, nebulosas).
- `PowerUpManager`: 3 tipos de power-ups (velocidad, escudo, maniobrabilidad).
- `ScoreSystem`: Puntuación por tiempo de supervivencia y objetos recolectados.
- `Renderer`: Dibujo de todos los elementos con estética pixel-art/CRT de los 80.
- `InputHandler`: Gestión de teclado (flechas) con respuesta instantánea.
- `AudioManager`: Efectos de sonido sintéticos via Web Audio API (sin archivos externos).

3. JUSTIFICACIÓN BASADA EN YAGNI

- ****¿Por qué NO microservicios/backend?*** El juego es 100% client-side. No hay persistencia de datos entre sesiones requerida en el MVP. YAGNI.
- ****¿Por qué NO múltiples archivos JS con módulos ES6?*** El entorno de ejecución es file:/// (archivo local). Los módulos ES6 nativos requieren servidor HTTP. Usar múltiples `<script>` sin módulos crea contaminación del scope global. El Namespace Pattern en un solo archivo resuelve ambos problemas elegantemente.
- ****¿Por qué NO un framework (Phaser.js, Three.js)?*** El requerimiento especifica HTML5/CSS/JS puro. Agregar una librería de 1MB+ viola KISS y YAGNI para un juego arcade 2D simple.
- ****¿Por qué NO Web Workers para física?*** La complejidad del juego (2D, ~20 objetos en pantalla) no justifica la overhead de comunicación entre workers. Un game loop de 60fps en el hilo principal es suficiente.
- ****¿Por qué NO archivos de imagen en assets/?*** Según restricción del skill, NO se incluyen binarios. Todos los gráficos se dibujan programáticamente en Canvas con formas geométricas y gradientes, logrando la estética retro de los 80 de forma nativa. Los sonidos se generan con Web Audio API (osciladores), eliminando la dependencia de archivos .mp3/.wav.

- **¿Por qué NO responsive.css separado?** Para un juego de canvas, la responsividad se maneja con JavaScript (resize del canvas). Un archivo CSS adicional para esto es overhead innecesario.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo 1 (BAJO):** Al tener todo JS en un archivo, el archivo `main.js` será extenso (~600-800 líneas). Mitigado con comentarios de sección claros y el Namespace Pattern.
- **Riesgo 2 (BAJO):** Sin archivos de audio externos, los efectos de sonido son sintéticos (beeps/bloops). Esto es en realidad una VENTAJA para la estética arcade de los 80.
- **Riesgo 3 (MEDIO):** Si en el futuro se requiere un leaderboard persistente, se necesitará agregar localStorage o un backend. Decisión diferida correctamente (YAGNI).
- **Riesgo 4 (BAJO):** El canvas no escala automáticamente en móviles. Mitigado con viewport meta tag y lógica de resize en JS.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- js/main.js | Estado: APROBADO | Iteraciones: 1
- css/styles.css | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 115,091

Tokens de salida (output): 31,085

Total tokens: 146,176

Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~15.3 horas-hombre ahorradas

"H squad de 4 personas × 2 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"2,529 input | !"1,538 output | 1 llamadas | ~1.7h ahorradas

MICROPLANIFICACION-DISEÑO: !"19,286 input | !"9,499 output | 4 llamadas | ~6h ahorradas

FRONTEND-DESARROLLO: !"37,621 input | !"15,518 output | 4 llamadas | ~4.2h ahorradas

test: !"55,655 input | !"4,530 output | 4 llamadas | ~3.4h ahorradas